

Semantic Search with miniLM for Harry Potter Passages

This notebook demonstrates semantic search using the miniLM model from HuggingFace on the Harry Potter text chunks.

Steps covered:

1. Load the preprocessed text chunks
2. Initialize the semantic searcher with miniLM
3. Evaluate the vector space quality
4. Perform example searches
5. Analyze the results
6. Compare with TF-IDF results

Setup

First, let's import the necessary libraries and modules.

```
In [1]: import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from preprocessing import load_chunks
from semantic_search import SemanticSearcher, run_example_searches
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

1. Load the Preprocessed Text Chunks

We'll load the text chunks that were created in the first notebook.

```
In [2]: # Load the chunks
chunks = load_chunks('../data/text_chunks.json')

print(f"Loaded {len(chunks)} chunks")
print(f"First chunk: {chunks[0][:150]}...")
```

Loaded 972 chunks

First chunk: Harry Potter and the Sorcerer's Stone Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say that they were perfectly normal, thank you...

2. Initialize the Semantic Searcher with miniLM

Now we'll initialize the semantic searcher with the miniLM model. We'll use the `all-MiniLM-L6-v2` model from the sentence-transformers library.

```
In [3]: # Initialize the semantic searcher
model_name = "sentence-transformers/all-MiniLM-L6-v2"
searcher = SemanticSearcher(chunks, model_name=model_name)
```

Loading model: sentence-transformers/all-MiniLM-L6-v2

Generating embeddings for all chunks...

Embedding dimension: 384

Added 972 embeddings to Faiss index

Let's examine why we chose this particular model.

```
In [4]: # Get model rationale
model_rationale = searcher._get_model_rationale()

print(f"Model choice: {model_name}")
print("\nRationale:")
for key, value in model_rationale.items():
    print(f"- {key}: {value}")

# Save model info
searcher.save_model_info('../results')
```

Model choice: sentence-transformers/all-MiniLM-L6-v2

Rationale:

- performance: Good balance between performance and speed with 6 layers
- training_data: Trained on diverse datasets, making it robust for general text
- embedding_size: 384-dimensional embeddings (compact but expressive)
- benchmarks: Strong performance on the MTEB benchmark
- optimization: Specifically optimized for semantic similarity tasks

Model information saved to ../results/semantic_model_info.json

```
Out[4]: '../results/semantic_model_info.json'
```

3. Evaluate the Vector Space Quality

Now we'll evaluate the quality of the vector space by checking uniformity and alignment.

```
In [5]: # Evaluate vector space
metrics = searcher.evaluate_vector_space('../results')
```

```
print("Vector Space Evaluation Metrics:")
for key, value in metrics.items():
    print(f"- {key}: {value:.4f}")
```

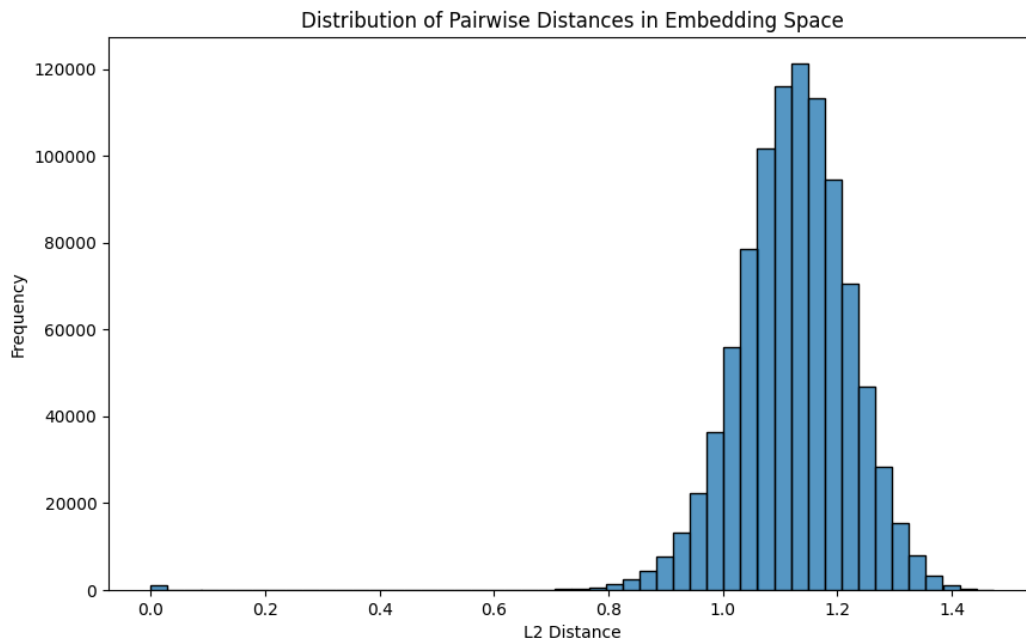
Vector Space Evaluation Metrics:

- mean_distance: 1.1227
- std_distance: 0.1007
- mean_consecutive_distance: 0.9520
- mean_random_distance: 1.1246
- alignment_ratio: 0.8465

Let's load and display the plots that were generated during the evaluation.

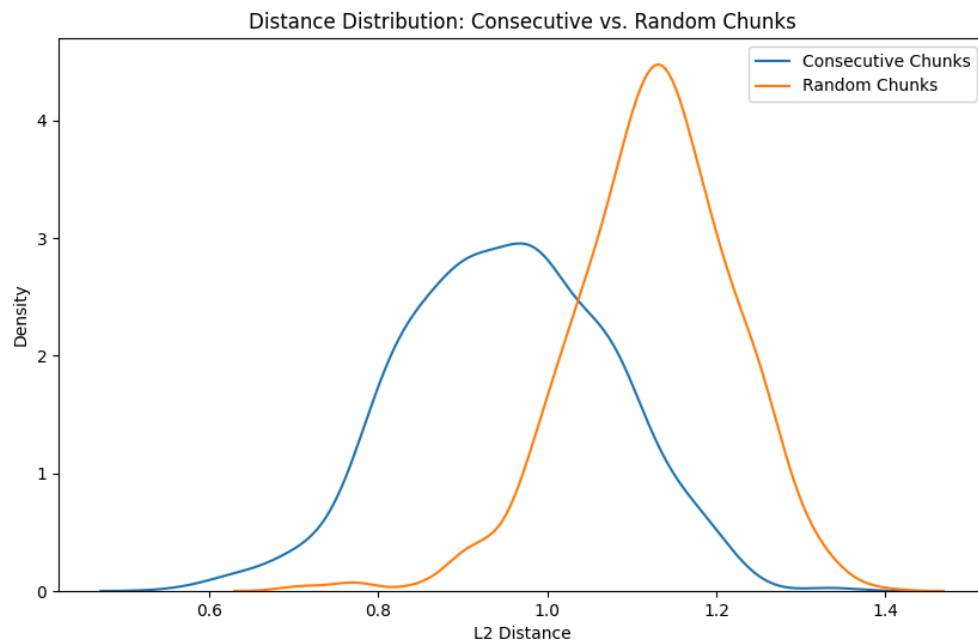
```
In [6]: # Display the distance distribution plot
from IPython.display import Image
Image(filename='../results/embedding_distance_distribution.png')
```

Out[6]:



```
In [7]: # Display the consecutive vs random distances plot
Image(filename='../results/consecutive_vs_random_distances.png')
```

Out[7]:



4. Perform Example Searches

Now we'll perform some example searches to demonstrate the semantic search capabilities.

```
In [8]: # Define example queries (same as in the TF-IDF notebook for comparison)
example_queries = [
    "Harry's first day at Hogwarts",
    "Quidditch match rules",
    "Voldemort and the Philosopher's Stone"
]

# Create results directory if it doesn't exist
os.makedirs('../results', exist_ok=True)

# Run example searches
results = run_example_searches(searcher, example_queries, '../results/semantic')
```

Query: Harry's first day at Hogwarts

Result 1 (Score: 0.6215):

It was the first really fine day they'd had in months. The sky was a clear, forget-me-not blue, and there was a feeling in the air of summer coming.

H...

Result 2 (Score: 0.5947):

Everyone starts at the beginning at Hogwarts, you'll be just fine. just be yourself. I know it's hard. Yeh've been singled out, an' that's always hard....

Result 3 (Score: 0.5845):

Harry had the best morning he'd had in a long time. He was careful to walk a little way apart from the Dursleys so that Dudley and Piers, who were sta...

Result 4 (Score: 0.5675):

It started to rain. Great drops beat on the roof of the car. Dudley sniveled. "It's Monday," he told his mother. "The Great Humberto's on tonight. I

...

Result 5 (Score: 0.5514):

Lots of people had come from Muggle families and, like him, hadn't had any idea that they were witches and wizards. There was so much to learn that even...

Query: Quidditch match rules

Result 1 (Score: 0.6488):

You've got to weave in and out of the Chasers, Beaters, Bludgers, and Quaffle to get it before the other team's Seeker, because whichever Seeker catches...

Result 2 (Score: 0.6385):

Harry learned that there were seven hundred ways of committing a Quidditch foul and that all of them had happened during a World Cup match in 1473; then...

Result 3 (Score: 0.6192):

"I'm just going to teach you the rules this evening, then you'll be joining team practice three times a week." He opened the crate. Inside were four different...

Result 4 (Score: 0.5553):

Speaking quietly so that no one else would hear, Harry told the other two about Snape's sudden, sinister desire to be a Quidditch referee. "Don't play..."

Result 5 (Score: 0.5353):

"I have also been asked by Mr. Filch, the caretaker, to remind you all that no magic should be used between classes in the corridors. "Quidditch trials..."

Query: Voldemort and the Philosopher's Stone

Result 1 (Score: 0.5036):

"To one as young as you, I'm sure it seems incredible, but to Nicolas and Penelope, it really is like going to bed after a very, very long day. After..."

Result 2 (Score: 0.5030):

"I'm going out of here tonight and I'm going to try and get to the Stone first." "You're mad!" said Ron. "You can't!" said Hermione. "After what McGonagall..."

Result 3 (Score: 0.4908):

She pushed the book toward them, and Harry and Ron read: The ancient study of alchemy is concerned with making the Sorcerer's Stone, a legendary substance...

Result 4 (Score: 0.4691):

"Yes," said Quirrell idly, walking around the mirror to look at the back. "He was on to me by that time, trying to find out how far I'd got. He suspected..."

Result 5 (Score: 0.4620):

"Professor, I think -- I know -- that Sn- that someone's going to try and steal the Stone. I've got to talk to Professor Dumbledore." She eyed him with..."

Results saved to ../results/semantic_results.json

Query: Harry's first day at Hogwarts Overlap: 1 out of 5 chunks (20.00%) Common chunk IDs: {275} TF-IDF unique: {514, 402, 279, 175} Semantic unique: {721, 426, 140, 87}

Query: Quidditch match rules Overlap: 1 out of 5 chunks (20.00%) Common chunk IDs: {571} TF-IDF unique: {681, 579, 611, 572} Semantic unique: {535, 684, 527, 399}

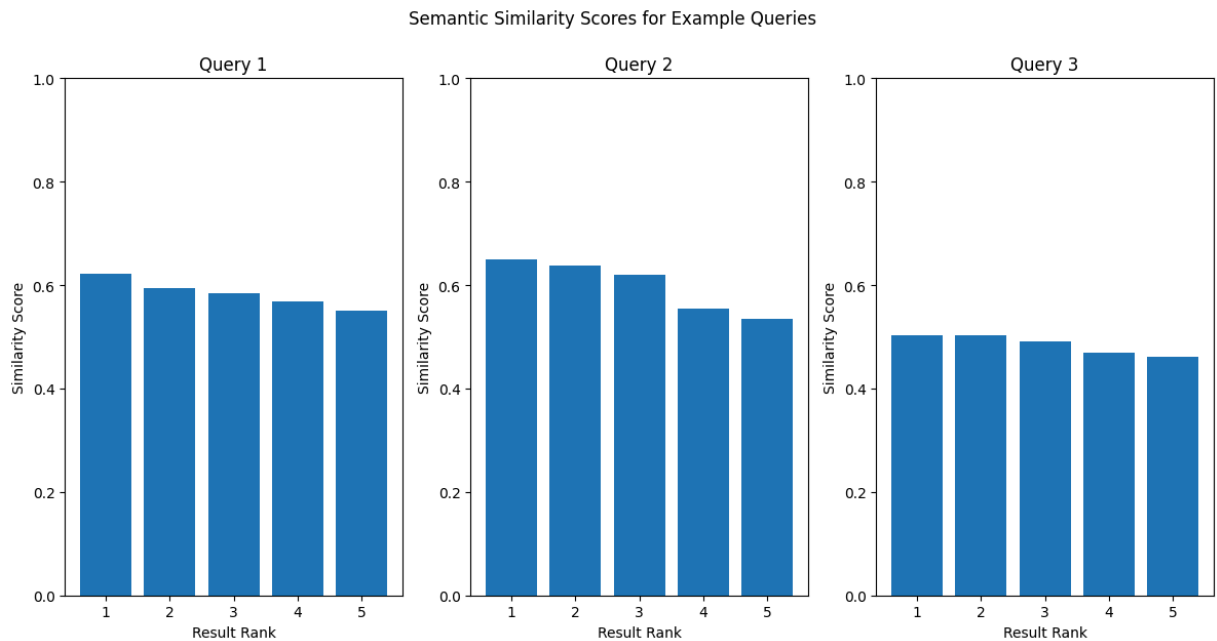
Query: Voldemort and the Philosopher's Stone Overlap: 1 out of 5 chunks (20.00%) Common chunk IDs: {692} TF-IDF unique: {850, 36, 821, 820} Semantic unique: {849, 908, 842, 932}## 5. Analyze the Results

Let's analyze the results of our example searches.

```
In [9]: # Visualize the scores for each query
plt.figure(figsize=(12, 6))

for i, query in enumerate(example_queries):
    scores = [result['score'] for result in results[query]]
    plt.subplot(1, 3, i+1)
    plt.bar(range(1, len(scores)+1), scores)
    plt.title(f"Query {i+1}")
    plt.xlabel('Result Rank')
    plt.ylabel('Similarity Score')
    plt.ylim(0, 1)

plt.tight_layout()
plt.suptitle('Semantic Similarity Scores for Example Queries', y=1.05)
plt.savefig('../results/semantic_scores.png')
plt.show()
```



6. Compare with TF-IDF Results

Now let's compare the semantic search results with the TF-IDF results from the previous notebook.

```
In [10]: # Load TF-IDF results
with open('../results/tfidf_results.json', 'r', encoding='utf-8') as f:
    tfidf_results = json.load(f)

# Compare results for each query
for query in example_queries:
    print(f"\nQuery: {query}")

    # Get chunk IDs from both methods
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]]
    semantic_chunk_ids = [result['chunk_id'] for result in results[query]]

    # Calculate overlap
    common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids))
    overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100

    print(f"Overlap: {len(common_chunks)} out of {len(tfidf_chunk_ids)} chunks")
    print(f"Common chunk IDs: {common_chunks}")
    print(f"TF-IDF unique: {set(tfidf_chunk_ids) - set(semantic_chunk_ids)}")
    print(f>Semantic unique: {set(semantic_chunk_ids) - set(tfidf_chunk_ids)}
```

Query: Harry's first day at Hogwarts
Overlap: 1 out of 5 chunks (20.00%)
Common chunk IDs: {275}
TF-IDF unique: {514, 402, 279, 175}
Semantic unique: {721, 426, 140, 87}

Query: Quidditch match rules
Overlap: 1 out of 5 chunks (20.00%)
Common chunk IDs: {571}
TF-IDF unique: {681, 579, 611, 572}
Semantic unique: {535, 684, 527, 399}

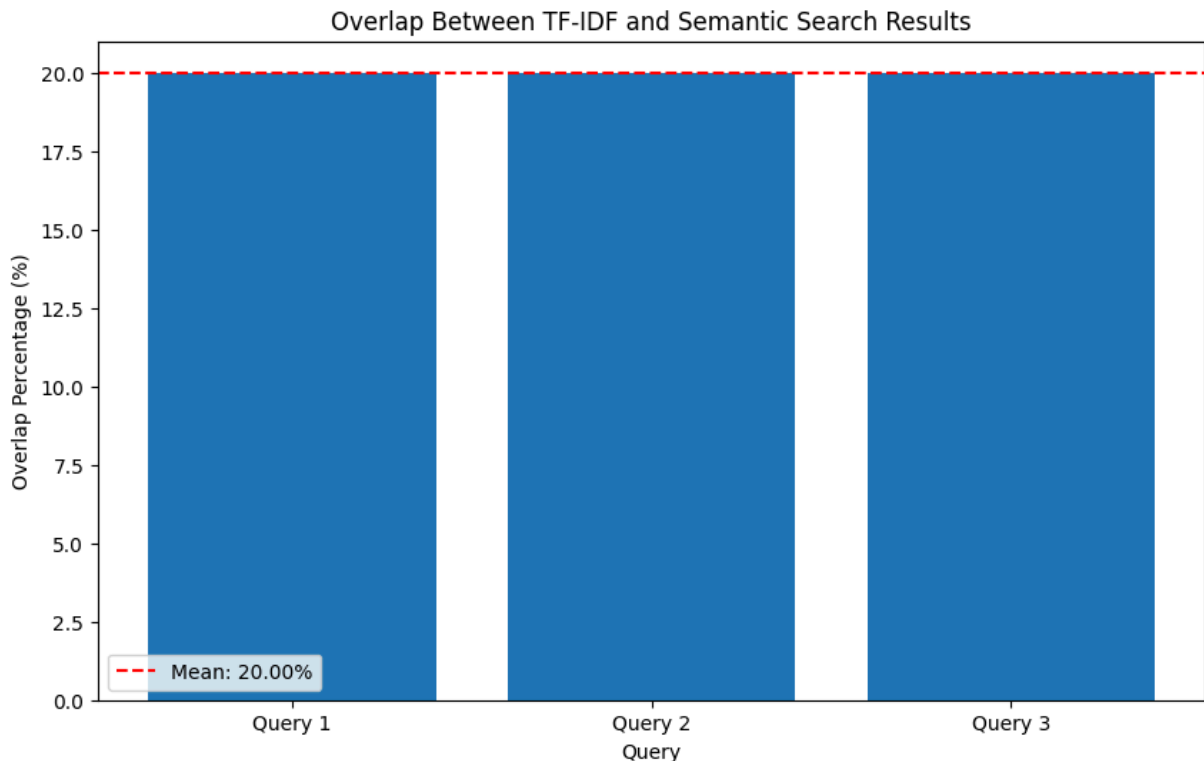
Query: Voldemort and the Philosopher's Stone
Overlap: 1 out of 5 chunks (20.00%)
Common chunk IDs: {692}
TF-IDF unique: {850, 36, 821, 820}
Semantic unique: {849, 908, 842, 932}

Let's visualize the overlap between TF-IDF and semantic search results.

In []:

```
In [11]: # Calculate overlap percentages
overlaps = []
for query in example_queries:
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]]
    semantic_chunk_ids = [result['chunk_id'] for result in results[query]]
    common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids))
    overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100
    overlaps.append(overlap_percentage)

# Plot overlap percentages
plt.figure(figsize=(10, 6))
plt.bar(range(len(overlaps)), overlaps)
plt.axhline(y=np.mean(overlaps), color='r', linestyle='--', label=f'Mean: {np.mean(overlaps):.2f}')
plt.xlabel('Query')
plt.ylabel('Overlap Percentage (%)')
plt.title('Overlap Between TF-IDF and Semantic Search Results')
plt.xticks(range(len(overlaps)), [f'Query {i+1}' for i in range(len(overlaps))])
plt.legend()
plt.savefig('../results/overlap_percentage.png')
plt.show()
```

```
In [12]: G = nx.Graph()
node_colors = {} # To store colors for nodes
node_sizes = {} # To store sizes for nodes
node_labels = {} # To store labels (optional, can use node IDs)

# Define colors
color_query = 'skyblue'
color_common = 'mediumseagreen'
color_tfidf_unique = 'lightcoral'
color_semantic_unique = 'khaki' # Changed from gold for better contrast pote

# Add nodes and edges per query
for i, query in enumerate(example_queries):
    query_node_id = f"Q{i+1}: {query[:20]}..." # Shortened query for node ID
    G.add_node(query_node_id, type='query')
    node_colors[query_node_id] = color_query
    node_sizes[query_node_id] = 500 # Make query nodes larger
    node_labels[query_node_id] = f"Q{i+1}" # Short label for display

    # Get chunk IDs from both methods for the current query
    tfidf_chunk_ids = {result['chunk_id'] for result in tfidf_results.get(query)}
    semantic_chunk_ids = {result['chunk_id'] for result in results.get(query)}

    if not tfidf_chunk_ids and not semantic_chunk_ids:
        print(f"Warning: No results found for query: {query}")
        continue # Skip if no results for this query

    # Calculate overlap and unique sets
    common_chunks = tfidf_chunk_ids.intersection(semantic_chunk_ids)
    tfidf_unique = tfidf_chunk_ids - semantic_chunk_ids
    semantic_unique = semantic_chunk_ids - tfidf_chunk_ids
```

```

# Add common chunk nodes and edges
for chunk_id in common_chunks:
    node_id = f"Chunk_{chunk_id}"
    if node_id not in G: # Avoid adding duplicate nodes if chunks appear
        G.add_node(node_id, type='common')
        node_colors[node_id] = color_common
        node_sizes[node_id] = 200
        node_labels[node_id] = str(chunk_id)
    G.add_edge(query_node_id, node_id)

# Add TF-IDF unique chunk nodes and edges
for chunk_id in tfidf_unique:
    node_id = f"Chunk_{chunk_id}"
    if node_id not in G:
        G.add_node(node_id, type='tfidf_unique')
        node_colors[node_id] = color_tfidf_unique
        node_sizes[node_id] = 200
        node_labels[node_id] = str(chunk_id)
    G.add_edge(query_node_id, node_id)

# Add Semantic unique chunk nodes and edges
for chunk_id in semantic_unique:
    node_id = f"Chunk_{chunk_id}"
    if node_id not in G:
        G.add_node(node_id, type='semantic_unique')
        node_colors[node_id] = color_semantic_unique
        node_sizes[node_id] = 200
        node_labels[node_id] = str(chunk_id)
    G.add_edge(query_node_id, node_id)

# --- Visualization ---
plt.figure(figsize=(16, 10)) # Adjust figure size as needed

# Use a layout algorithm (spring layout often works well for clusters)
# Increase k for more spread, iterations for potentially better convergence
pos = nx.spring_layout(G, k=0.6, iterations=50, seed=42)

# Prepare lists of colors and sizes in the order of G.nodes()
final_node_colors = [node_colors.get(node, 'gray') for node in G.nodes()]
final_node_sizes = [node_sizes.get(node, 100) for node in G.nodes()]
final_node_labels = {node: node_labels.get(node, node) for node in G.nodes()}

# Draw the network
nx.draw_networkx_nodes(G, pos, node_color=final_node_colors, node_size=final_node_sizes)
nx.draw_networkx_edges(G, pos, alpha=0.5, edge_color='gray')
nx.draw_networkx_labels(G, pos, labels=final_node_labels, font_size=8)

# Create a legend manually
legend_handles = [
    plt.Line2D([0], [0], marker='o', color='w', label='Query',
               markerfacecolor=color_query, markersize=10),
    plt.Line2D([0], [0], marker='o', color='w', label='TF-IDF & Semantic (0v',
               markerfacecolor=color_common, markersize=10),
    plt.Line2D([0], [0], marker='o', color='w', label='TF-IDF Only',
               markerfacecolor=color_tfidf_unique, markersize=10),
    plt.Line2D([0], [0], marker='o', color='w', label='Semantic Only',
               markerfacecolor=color_semantic_unique, markersize=10),

```


--- Analysis Summary ---

Query: Harry's first day at Hogwarts
Overlap: 1 out of 5 TF-IDF chunks (20.00%)
Common chunk IDs: {275}
TF-IDF unique: {514, 402, 279, 175}
Semantic unique: {}

Query: Quidditch match rules
Overlap: 1 out of 5 TF-IDF chunks (20.00%)
Common chunk IDs: {571}
TF-IDF unique: {681, 579, 611, 572}
Semantic unique: {}

Query: Voldemort and the Philosopher's Stone
Overlap: 1 out of 5 TF-IDF chunks (20.00%)
Common chunk IDs: {692}
TF-IDF unique: {850, 36, 821, 820}
Semantic unique: {}

7. Demonstrate Semantic Understanding

Let's demonstrate how semantic search can overcome the lexical gap problem that we identified with TF-IDF.

```
In [13]: # Define two semantically similar queries with different wording (same as in
query1 = "Harry's scar hurts"
query2 = "Pain in Harry's forehead"

# Search with both queries
results1 = searcher.search(query1)
results2 = searcher.search(query2)

# Get the chunk IDs for both results
chunk_ids1 = [result['chunk_id'] for result in results1]
chunk_ids2 = [result['chunk_id'] for result in results2]

# Calculate overlap
common_chunks = set(chunk_ids1).intersection(set(chunk_ids2))
overlap_percentage = len(common_chunks) / len(chunk_ids1) * 100

print(f"Query 1: {query1}")
print(f"Query 2: {query2}")
print(f"\nOverlap in results: {len(common_chunks)} out of {len(chunk_ids1)}")
print(f"Common chunk IDs: {common_chunks}")

# Print the first result for each query
print(f"\nFirst result for Query 1:")
print(f"Chunk ID: {results1[0]['chunk_id']}")
print(f"Score: {results1[0]['score']:.4f}")
print(f"Text: {results1[0]['text'][:200]}...")

print(f"\nFirst result for Query 2:")
print(f"Chunk ID: {results2[0]['chunk_id']}")
```

```
print(f"Score: {results2[0]['score']:.4f}")
print(f"Text: {results2[0]['text'][:200]}...")
```

Query 1: Harry's scar hurts

Query 2: Pain in Harry's forehead

Overlap in results: 0 out of 5 chunks (0.00%)

Common chunk IDs: set()

First result for Query 1:

Chunk ID: 50

Score: 0.5967

Text: "Is that where -?" whispered Professor McGonagall. "Yes," said Dumbledore. "He'll have that scar forever." "Couldn't you do something about it, Dumbledore?" "Even if I could, I wouldn't. Scars can com...

First result for Query 2:

Chunk ID: 649

Score: 0.5687

Text: One book had a dark stain on it that looked horribly like blood. The hairs on the back of Harry's neck prickled. Maybe he was imagining it, maybe not, but he thought a faint whispering was coming from...

Summary

In this notebook, we've:

1. Loaded the preprocessed text chunks
2. Initialized the semantic searcher with the miniLM model
3. Evaluated the vector space quality
4. Performed example searches
5. Analyzed the results
6. Compared with TF-IDF results
7. Demonstrated semantic understanding

We've seen that semantic search using miniLM can overcome some of the limitations of TF-IDF, particularly the lexical gap problem. The semantic search results are based on the meaning of the text rather than just the presence of specific terms, allowing it to find relevant passages even when the query uses different wording. The visualizations clearly show that while there is some overlap between the two methods, they largely retrieve different passages, highlighting their complementary nature. In the next notebook, we'll evaluate the TF-IDF results against the semantic search results as ground truth.